

Lesson 9, Part 2 Activity: Estimate One Slice of Your Project

CS 5388 / CS 4390 Software Project Management | Team activity, complete together

Team: 3 Project: SkillUp

What this activity is for

Your Sprint 2 business case has to answer a question every sponsor asks: how much work would this take, and what would it cost? You are not going to build the system in this course. You are doing what a project manager does before a project is approved: estimating the work so a decision can be made.

Estimating a whole project at once hides mistakes. So you will start small. Pick one or two major parts of your project, called a slice, and estimate only that slice. A small slice lets you count everything, check every number, and see clearly where your assumptions matter.

You will estimate the same slice twice, using two methods that start from different information. Then you will compare the two answers, explain why they differ, and present a range with your assumptions written down. That range becomes the Estimation Appendix on your Sprint 2 portal page.

What to bring

1. Your Release 1 capability list (Step 1 of the Part 1 starter).
2. Your backlog from the Part 1 activity: user stories, MoSCoW categories, and planning poker point sizes.
3. Your similar-products research from Part 1 (small and big products, with timelines and team sizes).
4. The Part 2 slides, especially Slides 13 to 17 (sizing), 19 to 24 (team size), 25 to 27 (story points), and 31 to 33 (reconciling).

How the activity fits together

Step	What you do	What you end up with
1	Pick your slice	One or two major parts of your project, with their stories, screens, and data listed
2	Choose two estimation methods	Two methods that use different starting information
3	Estimate the slice with each method	Two effort estimates, both in team hours
4	Check the estimates for team size	Confidence that coordination costs are included
5	Compare the two estimates	A written explanation of why they differ
6	Present the result	A dated range, in hours, dollars, and weeks, ready for the business case

Key terms

Term	What it means in this activity
Slice	One or two major parts of your project that a user would recognize, such as “buy a permit.” You estimate only this.
Size	How much software the slice contains, measured in function points, use-case points, or story points.
Effort	Total team hours of work. If 5 people each work 10 hours, that is 50 hours of effort.
Productivity rate	Hours of effort per unit of size, such as 8 hours per function point. The rate drives the whole estimate.
Assumption	Any number you chose rather than measured. Label it, so a reader knows it could change.
Velocity	Story points a team completes per sprint. You have no sprint history yet, so yours is an assumption.
Loaded rate	The full hourly cost of one person: salary plus benefits, overhead, and tools.
Range	A low and a high estimate. Honest estimates are ranges, not single numbers.
O, M, P	Optimistic, most likely, and pessimistic estimates, used in the PERT formula $(O + 4M + P) \div 6$.

About the examples

Every italic example uses the same slice from the fictional Campus Parking System: “Buy a semester permit.” Following one slice through every method shows how the pieces connect. Your own numbers come from your project.

Step 1. Pick your slice

A good slice is a part of your project that a user would recognize and that is useful on its own. It usually covers 3 to 8 user stories from your backlog.

How to choose

1. Start with a Must-have capability that is central to your project. If it did not exist, the project would not make sense.
2. If you pick a second part, make it different in kind, such as one part users interact with and one part that produces reports or connects to another system.
3. Do not pick the easiest part. The slice should include some real complexity, such as payments, another system, or stored data.
4. Write down what is out of the slice, so your estimate has a clear boundary.

Question	Our slice
Slice name (one or two major parts)	User Portfolio Management and Analysis
Why we chose it	This slice is our core use system that glues together every other critical subsystem in our project for it to work together to achieve the projects' goal.
Screens or forms where users enter data	User will have a portfolio where they can see all their activities. Users will have a screen to record new recreational activities.
Reports, results, or lookups users see	Report on progress per activity. Users can look up their past reports.
Data the slice stores	Recreational Activities (photos, description, date/time, duration)
Other systems it connects to	Certificates, instructors
Out of the slice (not included)	Social Aspect Slice

Stories in the slice

User story	MoSCoW	Points
As a user, I want to record my recreational activities so that I can track my journey.	Must	5
As a user, I want to track my progress for individual recreational activities so that I can see my improvement over time.	Must	13
As a user, I want to view my progress and accomplishments across all of my recreational activities in one place so that I do not have to track them separately.	Should	5
As a user, I want to customize my portfolio so that I can personalize it to fit my preferences.	Could	3
As a user, I want to set personalized goals so that I can track my progress toward achieving them.	Should	5

User story	MoSCoW	Points
Total points		31

Example slice, “Buy a semester permit”: a student registers a vehicle (3 points), chooses a lot and permit type (5), pays online (8), receives a digital permit by email (3), and looks up the permit later (2), for 21 points. It stores permit and vehicle records and connects to the student registry and a payment service. Citations and appeals are out of the slice.

Step 2. Choose two methods

Each method starts from different information. Choose two that do not share the same starting point, so that if they agree, the agreement means something (Part 2, Slide 31).

Method	It starts from	A good choice if	Watch out for
Analogous	Similar products and how long they took	You found at least one product close to yours	Copying a big company's timeline without adjusting
Function points	Counts of what users enter, see, and store	Your Step 1 table lists screens, results, and data	A productivity rate with no source
Use-case points	The actors and use cases in your slice	You can list who uses the slice and for what	Default adjustment factors used without saying so
Story points to cost	Your planning poker points and a velocity	Every story in the slice has points	Velocity is a guess until you run sprints

Suggested pairs

1. Function points plus analogous: one starts from your own design, the other from the market. The examples below use this pair.
2. Story points plus analogous: one starts from your backlog, the other from the market.
3. Function points plus story points: one starts from system features, the other from team judgment of story size.

Our two methods: 1. Analogous 2. Story points to cost

Why these two use different starting information: Analogous uses estimates from a similar real product, while Story points to cost is based on our own estimate of the project's work. One uses real data while the other is an assumption of our team's effort and velocity.

Step 3. Estimate the slice

Story Points:

Input	Value	Note
Slice points	31	From your Step 1 story table
Team size	5	Five or six at most
Hours per person per sprint	40	State your assumption
Velocity: low / middle / high	15 / 19 / 25	Assumptions until you have sprint data
Sprints needed (low to high velocity)	2.07 / 1.63 / 1.24	Slice points ÷ velocity
Effort in hours (low to high)	414 / 326 / 248	Sprints × team size × hours per person per sprint

3D. Analogous (Part 2, Slide 10)

What it measures: effort based on how long a similar real product took. You turn the product's timeline into person-months, scale it down to the size of your slice, then convert it to hours.

How to do it

1. Pick the similar product closest to your project from your Part 1 research, and note its team size and time to first release.
2. Multiply team size by months to get person-months. This is the effort the comparable product took.
3. Estimate what fraction of their first release your slice matches. If your slice is about a tenth of what they shipped, multiply by 0.1. Give a low and high fraction and explain them.
4. Adjust for team size. A much larger team spent more of its time coordinating, so its effort per feature is higher than yours will be.
5. Convert to hours. State the hours per person-month you assume (for example, 160).

Question	Answer
Comparable product, team size, and time to first release	Everyday.app, 1 developer, approximately 3 weeks to first working version.
Person-months (team size $\times$ months)	$1 * \frac{3}{4} = 0.75$
Fraction of their release our slice matches (low to high), and why	Low: 1, High: 1.5. Our slice contains the core functionality of Everyday's initial release which includes recording activities and viewing progress, but it adds functionality such as progress by individual recreational activity, a portfolio with customization, and personalized goals. Because our slice contains more features than Everyday's initial release, we are doing all (1) or more (1.5). We estimate our slice to be the same size and 50% larger than Everyday.
Team size adjustment and why	We plan to add 20% more time for coordination due to having 5 team members compared to the individual developer.
Adjusted effort in hours (low to high)	Low: $0.75 * 1 = .75$ person-months High: $.75 * 1.5 = 1.125$ person-months Assuming 160 hours per person month Low: $0.75 * 160 = 120$ hours High: $1.125 * 160 = 180$ hours Apply 20% for coordination Low: 144 hours High: 216 hours
Confidence (low, medium, high) and why	Low confidence because Everyday.app is similar to the core functionality of our slice. However, its initial release was developed by one person, while our team has five members, and our slice includes additional functionality. The 20% coordination adjustment and the estimated slice size are assumptions, so the final effort could vary.

Example: ParkEasy (fictional) took 4 people about 9 months, or 36 person-months, to launch a map of lots and pay-by-phone in one city. Buying a permit is roughly 8 to 12 percent of that release. 36×0.08 to $0.12 = 2.9$ to 4.3 person-months, or about 460 to 690 hours at 160 hours per person-month (about 576 hours at 10 percent).

Before you move on: put both estimates in hours

Step 5 compares your two estimates directly, so both must be in team hours for the same slice. If an estimate came out in months, sprints, or dollars, convert it now and show the conversion.

Step 4. Check your estimates for team size (Part 2, Slides 19 to 24)

Larger teams spend more of their time coordinating: meetings, handoffs, and fitting work back together. Estimators tend to underweight this cost, so check each estimate against the list below. Anything marked No needs a fix or a written explanation.

Check	Yes / No	Why it matters
Our team is five or six people at most.	Yes	Beyond six, split into two teams.
We counted our communication paths: $n(n - 1) \div 2 =$ __10__	Yes	Five people share 10 paths; ten share 45.
We added explicit work for integrating the pieces of the slice.	Yes	Breaking work into pieces hides the cost of putting it back together.
We did not assume that more people means proportionally more output.	Yes	Coordination grows faster than headcount.
We estimated for the team we have.	Yes	Not for a larger team we wish we had.
We adjusted, not copied, timelines from much larger teams.	Yes	Their effort includes their coordination costs.

Step 5. Compare your two estimates (Part 2, Slides 31 and 32)

Two estimates almost never match. The gap is useful: it points to the assumption you are least sure about. Your job is to find that assumption and name it.

	Method 1	Method 2
Method	Analogous	Story Points
Estimate in hours (low to high)	144 hrs. - 216 hrs.	248 hrs - 414 hrs
Middle value in hours	180 hours	326 Hours
Input that drives it most	The slice compared with the Everyday app	Velocity (low, middle, high)
Is that input measured or assumed?	Assumed	Assumed, we do not have sprint data to help measure.

Gap: larger middle value $\div$ smaller middle value = **1.81**

If one estimate is more than about twice the other, recheck your counts and conversions before explaining the gap.

Questions to answer

1. Do your two methods depend on the same assumption? If so, their agreement proves little.

No they start from different information

2. Which single input moves the answer most? Try changing it and see.
Assumed velocity seems to be a major contributor to estimate
3. What information would narrow the gap, and when could you get it?
Knowing the actual team velocity once we finish the first sprint

Our explanation of the gap (one paragraph)

Both methods depend on different information. Analogue mainly focuses on the Everyday.app and how it compares to our time slice while Story Points mainly has its own story points and assumed velocity. For now, we don't have sprint data yet, so we don't have an answer to that yet. Once our team completes a sprint, we can use our actual velocity to make a more accurate estimation and help narrow done gaps between the two methods.

Example: function points gave a middle value of 472 hours and the analogous method gave 576 hours, a gap of about 1.2. The two are fairly close, and they start from different information, so the agreement is meaningful. The largest uncertainty is the productivity rate, which is our assumption, and the fraction of ParkEasy's release that our slice represents.

Step 6. Present the estimate (Part 2, Slide 33)

This is the table your sponsor reads. It gives a range instead of a single number, shows what the range depends on, and is dated because estimates change as you learn more. It becomes the Estimation Appendix on your Sprint 2 portal page.

How to fill it in

1. Range: the lowest and highest hours across both methods.
2. Expected value: set O to your low end and P to your high end. Choose M, your most likely value, somewhere between them and explain why. Then calculate $(O + 4M + P) \div 6$.
3. Cost: multiply hours by a loaded hourly rate. State the rate and where it came from, such as published wage data plus an overhead percentage you choose.
4. Calendar time: divide hours by (team size $\times$ productive hours per person per week). State the productive hours you assume.

Field	Our estimate
Slice estimated	User Portfolio Management and Analysis
Range in hours (low to high)	144 – 414 hrs.
Expected hours: $(O + 4M + P) \div 6$, and why we chose M	$(144 + 4(253) + 414) / 6 = 262$ hrs. We chose M as 253 because it's the middle average between the two middle values of both methods.
Cost range and loaded rate used	\$60/hour We estimated this from the average market value of a software engineer. Cost range would be \$7,200 low, \$13,100 expected, and \$20,700 high.
Calendar estimate in weeks, and hours per week assumed	Assuming 5 team members $\times$ 20 hours per person per week = 100 team hours/week, the range is approximately 1.4–4.1 weeks, with the expected 262 hours taking approximately 2.6 weeks.
Methods used	Analogous and story points
Key assumptions	The 20% additional effort Story Point velocity is assumed at 15/19/25 points per sprint Each members works 40 hours per week
Team size assumed	5
What would change this estimate	Changes to the slice or scope, actual team velocity of each sprint
Date of this estimate	09/29/26

Example, “Buy a semester permit” slice: range 354 to 708 hours. With $M = 520$, expected hours = $(354 + 2,080 + 708) \div 6 \approx 524$. At an assumed loaded rate of \$60 per hour, cost is about \$21,000 to \$42,500. With 5 people at an assumed 30 productive hours a week (150 team hours), the expected 524 hours is about 3.5 weeks.

Going further (optional)

Once your slice estimate is solid, you can use it to make a rough estimate of the whole project. If your slice holds about one fifth of your backlog points, the whole project is roughly five times the slice. Treat that number as a first rough guess, and repeat this activity on more slices as you learn more.

Before you submit

1. Your slice is named, with its stories, screens, data, and boundary written down.
2. Both estimates are for the same slice and in hours, with the conversions shown.
3. Every assumed number is labeled as an assumption.
4. Every sourced number has a source you can link.
5. The team-size check is complete, and any No has a fix or explanation.
6. The final estimate is a dated range, not a single number.
7. The individual estimation memo is your own work, written without AI.